



Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\{new\}} = g_{\{current\}} + 1$, $h_{\{new\}}$ using your heuristic, and $f_{\{new\}} = g_{\{new\}} + h_{\{new\}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.
5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS orders its queue by $g(n)$ alone. The cost already spent reaching n . It is backward-looking and expands outward uniformly in all directions, ignoring where the goal is. A* orders by $f(n) = g(n) + h(n)$, adding an estimate of the remaining cost. Among nodes that cost the same to reach, it expands the one that looks closer to the goal, so the search stretches toward the goal and expands far fewer nodes for the same optimal path. UCS is just A* with $h(n) = 0$.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

A heuristic is admissible if it never overestimates the true remaining cost. With 4-way movement, each move changes one coordinate by 1 at cost 1, so reaching the goal requires at least $|\Delta x| + |\Delta y|$ moves, exactly Manhattan distance. That is the cost on an empty grid; walls can only force detours and increase the real cost, never reduce it. So $h(n) \leq \text{true cost}$ always, and Manhattan is admissible. If it were inadmissible, A* would lose its optimality guarantee: an overestimate inflates f for a node on the true shortest path, burying it in the queue, so a goal found via a worse path can be returned first. A* would still find a path, often faster, but not the shortest one.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No. With diagonals, one move can reduce both $|\Delta x|$ and $|\Delta y|$ at once, so $(0,0) \rightarrow (3,4)$ truly costs $\max(3,4) = 4$ while Manhattan returns 7, an overestimate, so it is no longer admissible and A* would return suboptimal paths. Switch to **Chebyshev distance**, $h = \max(|\Delta x|, |\Delta y|)$, if a diagonal costs 1; use **octile distance** if it costs $\sqrt{2}$. Euclidean stays admissible in both cases but is weaker. It underestimates more and expands more nodes.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Nearest-pellet distance is weak because it ignores all food beyond the first, collapsing to near zero even when many pellets remain. A stronger heuristic treats the state as (agent position, remaining food set) and estimates the cost of clearing all of it: $h(n) = \text{distance}(\text{agent} \rightarrow \text{nearest pellet}) + \text{MST}(\text{remaining pellets})$ using Manhattan or true maze distance as edge weights. It is admissible: any solution must travel to some first pellet, then form a connected route over all remaining pellets, which costs at least the minimum spanning tree. It stays large while food is scattered and shrinks

only as food is eaten, so A* clears clusters efficiently instead of greedily chasing the nearest pellet and backtracking. The cost is $O(k^2 \log k)$ per evaluation, so cache it per food-set. A cheaper admissible fallback is $h = \max$ distance to any remaining pellet, since every pellet must eventually be reached.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

**Incomplete: `{filledCount}/{fields.length}` questions answered.
Please provide detailed responses for all questions.**



FACULTY OF COMPUTING